

1. Introduction

Modern speech processing systems have achieved high accuracy in **monolingual environments**. However, they often struggle with **code-switched speech**, where multiple languages are used within the same utterance. This phenomenon is especially common in multilingual regions such as India, where languages like Hindi and English are frequently mixed (e.g., Hinglish in classrooms).

This assignment focuses on designing and implementing an **end-to-end pipeline** capable of processing code-switched speech and synthesizing it into a **Low-Resource Language (LRL)**.

The key objective is to develop a **modular and functional system** that performs the following tasks:

- Audio preprocessing
- Speech-to-text (STT) conversion
- Language identification (LID)
- Phonetic (IPA) conversion
- Translation
- Speech synthesis (TTS)
- Robustness evaluation

Due to computational limitations and lack of large-scale datasets, this work emphasizes **system integration, modularity, and pipeline functionality** rather than extensive model training.

2. System Architecture

The system follows a **modular pipeline architecture**, where each component is independently executable and reusable.

Pipeline Flow:

Audio → Preprocessing → STT → LID → IPA Conversion → Translation → TTS → Evaluation

Each module is designed to:

- Operate independently
- Allow easy debugging and replacement
- Support scalability for future improvements

3. Data Preparation

3.1 Input Data

- **Source:** 10-minute classroom speech segment
- **Format:** WAV
- **Sampling Rate:** 16 kHz

Files Used:

- `data/samples/original_segment.wav`
- `data/samples/student_voice_ref.wav`

3.2 Output Files

The pipeline generates the following outputs:

- `outputs/clean_segment.wav` — Noise-reduced audio
- `outputs/transcript.txt` — STT output
- `outputs/transcript_ipa.txt` — Phonetic transcription
- `outputs/translated_lrl.txt` — Translated text (LRL)
- `outputs/output_LRL_cloned.wav` — Synthesized speech
- `outputs/lid_labels.txt` — Language labels
- `outputs/evaluation_metrics.json` — Performance metrics

4. Audio Preprocessing

4.1 Method: Spectral Subtraction

To improve transcription quality, **noise reduction** was performed using the spectral subtraction technique.

Steps:

1. Convert audio signal into frequency domain using **Short-Time Fourier Transform (STFT)**
2. Estimate background noise from initial silent frames
3. Subtract noise magnitude from the signal
4. Reconstruct cleaned signal using **Inverse STFT (ISTFT)**

4.2 Mathematical Formulation

Let:

- $(X(f,t))$ = Noisy signal
- $(N(f))$ = Estimated noise

The cleaned signal is computed as:

$$S(f,t) = \max(|X(f,t)| - \alpha N(f), \beta)$$

Where:

- (α) = Noise scaling factor
- (β) = Minimum spectral floor

4.3 Results

- Significant noise reduction achieved
- Signal amplitude normalized
- Output saved as `clean_segment.wav`

5. Speech-to-Text (STT)

5.1 Model Used

Speech recognition was implemented using **OpenAI Whisper**.

Reasons for selection:

- Robust to noisy inputs
- Supports multilingual transcription
- Pre-trained and easy to integrate

5.2 Process

- Cleaned audio is passed to the Whisper model
- Model generates textual transcription
- Output stored in `transcript.txt`

5.3 Observations

- Transcription is generally coherent
- Output is predominantly English with limited code-switching
- Minor transcription errors observed due to accents and overlap

6. Language Identification (LID)

6.1 Approach

Due to the absence of labeled training data, a **rule-based LID system** was implemented:

- English words → labeled as `en`
- Non-English words → labeled as `hi`

6.2 Limitations

- Not performed at frame or phoneme level
- No machine learning model used
- Reduced accuracy for ambiguous or mixed words

6.3 Output

- Language labels stored in `lid_labels.txt`

7. IPA Conversion

7.1 Motivation

Standard **Grapheme-to-Phoneme (G2P)** models do not perform well on code-switched text. To address this, a **custom phonetic mapping layer** was implemented.

7.2 Approach

- Map English words to phonetic representations
- Handle Hindi transliterations manually
- Apply rule-based phonetic transformations

7.3 Output

- Phonetic transcription stored in `transcript_ipa.txt`

8. Translation to Low-Resource Language

8.1 Approach

A **dictionary-based translation method** was used:

```
[  
T(word) =  
\begin{cases}  
\text{mapped word}, & \text{if exists in dictionary} \\  
\text{original word}, & \text{otherwise}  
\end{cases}  
]
```

This approach ensures:

- Simplicity
- Low computational overhead
- Partial preservation of meaning

8.2 Limitations

- Limited vocabulary coverage
- No contextual understanding
- Cannot handle complex sentence structures